DRIVEMATE AI

Driving School AI Chatbot

Implementation Plan & Development Roadmap

Project type: Full-stack AI-powered web application

Target: Sri Lankan driving schools | **Languages:** Sinhala and English

PROJECT GOAL

Build a helpful virtual assistant that answers driving-school questions using verified information, supports customer enquiries, and collects lesson-booking requests. The project will be developed in clear stages, with testing after each major feature.

Beginner-friendly guide • Architecture • Technologies • Implementation phases • Testing • Deployment

1. Project Overview

DriveMate AI is a web-based chatbot for a Sri Lankan driving school. Customers can ask questions in everyday language, learn about courses and registration, and submit requests for driving lessons.

Core objectives

- Understand natural-language questions from customers.
- Answer from the driving school's approved information rather than guessing.
- Support Sinhala and English conversations.
- Explain courses, fees, registration steps, required documents and policies.
- Collect customer enquiries and lesson-booking requests.
- Keep relevant conversation context during a chat.
- Provide a responsive interface for desktop and mobile.

2. Main System Architecture

Layer	Suggested technology	Responsibility
Frontend	Next.js + TypeScript	Chat window, messages, language selector and user interface.
Styling	Tailwind CSS	Responsive layout and visual design.
Backend	Next.js Route Handlers	Receives requests, validates input and runs application logic.
AI model	OpenAI API	Interprets questions and generates responses.
Knowledge retrieval	RAG + vector search	Finds relevant information from school documents.
Database	PostgreSQL	Stores enquiries, booking requests and conversation records.
Deployment	Vercel + managed database	Publishes the application and hosts its data.

How a message travels: The customer types a question in the frontend. The frontend sends it to a backend API route. The backend retrieves relevant school information, sends the question and context to the AI model, receives the answer, and returns it to the frontend for display.

3. Technologies and Their Purpose

Technology	Why it is used
HTML	Defines the structure of web content.
CSS / Tailwind CSS	Controls appearance, spacing and responsive design.
JavaScript	Adds behaviour and application logic.
TypeScript	Adds types to JavaScript to catch many mistakes earlier.
Next.js	Provides the web application, pages and backend API routes.
OpenAI API	Provides an AI model for natural-language responses.
PostgreSQL	Stores structured application data.
Vector database	Supports semantic search over document chunks for RAG.

Technology	Why it is used
Git / GitHub	Tracks code changes and stores the project repository.
Vercel	Deploys the Next.js application.

4. Development Phases

Phase 1 — Planning and environment setup

- Confirm the project purpose, target users and required features.
- Install Visual Studio Code, Node.js and Git.
- Create a Next.js project with TypeScript, ESLint, Tailwind CSS and App Router.
- Create a GitHub repository and confirm the development server runs.

Deliverable: a working local development environment.

Phase 2 — UI/UX and frontend

- Design the chatbot homepage and conversation layout.
- Build message bubbles, input field, send button and suggested questions.
- Add language selection and responsive mobile styling.
- Use mock responses first so the interface can be tested before AI integration.

Deliverable: a working chat interface using sample responses.

Phase 3 — Backend development

- Create API routes for chat, booking requests and enquiries.
- Validate incoming messages and form fields.
- Handle errors and define a consistent response format.
- Add basic conversation/session handling.

Deliverable: frontend can send messages to the backend.

Phase 4 — AI integration

- Create an API key with the chosen AI provider.
- Store the key in a server-side environment variable.
- Create system instructions that define the assistant's role and limits.
- Send user messages to the model and display its response.
- Pass conversation context when appropriate.

Deliverable: the chatbot can hold an AI-powered conversation.

Phase 5 — Knowledge base and RAG

- Gather approved course, fee, registration, document and FAQ information.
- Extract text from documents and divide it into smaller chunks.
- Create embeddings for the chunks and store them in a vector database.
- Search for relevant chunks when a user asks a question.
- Give retrieved information to the AI model as context.
- Require the chatbot to say when the information is unavailable.

Deliverable: answers grounded in the driving school's supplied information.

Phase 6 — Database and booking requests

- Design PostgreSQL tables for users, courses, bookings, conversations, messages and enquiries.
- Create validated booking-request forms or guided chat steps.
- Store customer requests securely.
- Show that a request was submitted, not that a lesson is confirmed until staff verify availability.

Deliverable: persistent customer enquiries and booking requests.

Phase 7 — Testing and evaluation

- Test common English and Sinhala questions.
- Test wrong, unclear and unsupported questions.
- Check that fees and policies match the approved source data.
- Test booking validation, error handling and mobile layouts.
- Measure answer correctness, retrieval relevance and response time.

Deliverable: tested application with documented issues and improvements.

Phase 8 — Deployment

- Push the finished code to GitHub.
- Configure production environment variables.
- Deploy the Next.js application to Vercel.
- Connect the production database and verify permissions.
- Test the live application on desktop and mobile.
- Add a custom domain if one is available.

Deliverable: publicly accessible chatbot.

5. RAG: How the Chatbot Uses School Information

RAG means Retrieval-Augmented Generation. It combines information retrieval with an AI language model. Instead of expecting the model to know the school's current fees or policies, the application searches the school's own information and provides relevant passages to the model before it answers.

Step	What happens
1. Prepare documents	Collect approved PDFs, FAQs, course details and policies.
2. Extract and split	Extract text and divide it into manageable chunks.
3. Create embeddings	Convert each text chunk into a numerical representation of its meaning.
4. Store	Save embeddings and text in a vector database.
5. Retrieve	Find the most relevant chunks for a customer's question.
6. Generate	Give the question and retrieved context to the AI model to create an answer.

Reliability rule: The chatbot must not invent prices, lesson availability, licensing requirements or school policies. If the knowledge base does not contain a reliable answer, it should say so and direct the customer to the school.

6. Proposed Database Tables

Table	Example fields
users	id, name, phone, email, created_at
courses	id, course_name, description, fee, active
bookings	id, user_id, course_id, preferred_date, status
conversations	id, session_id, created_at
messages	id, conversation_id, role, content, created_at
enquiries	id, user_id, enquiry_type, message, status

Only collect the personal information needed to handle the request. Restrict access to customer records and define a retention policy.

7. Suggested Project Structure

```
driving-school-chatbot/
■■■ public/
■■■ src/
■ ■■■ app/
■ ■ ■■■ api/
■ ■ ■ ■■■ chat/route.ts
■ ■ ■ ■■■ booking/route.ts
■ ■ ■ ■■■ enquiries/route.ts
■ ■ ■■■ page.tsx
■ ■ ■■■ layout.tsx
■ ■ ■■■ globals.css
■ ■■■ components/
■ ■ ■■■ ChatWindow.tsx
■ ■ ■■■ MessageBubble.tsx
■ ■ ■■■ ChatInput.tsx
■ ■ ■■■ SuggestedQuestions.tsx
■ ■■■ lib/
■ ■ ■■■ ai.ts
■ ■ ■■■ database.ts
■ ■ ■■■ rag.ts
■ ■■■ types/
■ ■ ■■■ chat.ts
■ ■■■ data/
■ ■■■ driving-school.ts
■■■ .env.local
■■■ package.json
■■■ tsconfig.json
```

8. Testing and Performance Evaluation

Test area	Example test	Expected result
Language	Ask a question in Sinhala and English.	Understands and replies in the selected language.
Correctness	Ask for a course fee.	Returns the approved fee or says it is not available.
Relevance	Ask an unrelated question.	Explains its scope and avoids pretending to know.
Context	Ask a follow-up question.	Uses relevant previous conversation context.
Booking	Submit a valid lesson request.	Validates and saves the request.
Validation	Submit missing or invalid fields.	Shows a clear error and does not save invalid data.
Responsive UI	Open the chat on a phone.	Layout remains usable.

Useful evaluation measures

- Answer correctness: proportion of test questions answered correctly.
- Retrieval relevance: whether retrieved document passages support the answer.
- Response time: time taken to return a response.
- Fallback quality: whether unknown questions are handled honestly.
- Booking success: proportion of valid requests successfully stored.

9. Security and Reliability

- Keep AI API keys on the server; never put secrets in browser code.
- Use environment variables and do not commit .env.local to GitHub.
- Validate and limit user input on the server.
- Use rate limiting to reduce spam and unexpected API costs.
- Protect customer data with appropriate access controls.
- Use secure database credentials and production permissions.
- Do not claim a lesson is confirmed until staff verify availability.
- Log errors without unnecessarily logging sensitive customer data.

10. Suggested Six-Week Schedule

Week	Main focus	Milestone
1	Planning and environment setup	Next.js app runs locally.
2	UI/UX and frontend	Chat interface works with mock data.
3	Backend and AI integration	Chat sends messages and receives AI responses.
4	Knowledge base and RAG	Answers use school documents.
5	Database and booking requests	Enquiries and requests are stored.
6	Testing and deployment	Live application is tested.

The schedule is an estimate for consistent part-time work. The actual duration depends on coding experience, the quality of school information and integration issues.

11. Final Deliverables

- Responsive driving school chatbot website.
- AI-powered English and Sinhala conversation.
- RAG-based school information retrieval.
- Course and FAQ knowledge base.
- Customer enquiry and lesson-booking request workflow.
- PostgreSQL database integration.
- Test cases and evaluation report.
- Deployed application and source-code repository.

12. Recommended Starting Checklist

- Install Visual Studio Code, Node.js LTS and Git.
- Create a Next.js project using TypeScript, ESLint, Tailwind CSS and App Router.
- Run the development server and open localhost:3000.
- Create a GitHub repository.
- Build the chatbot interface with mock messages before connecting AI.
- Collect real driving school information before implementing RAG.

First project command

```
npx create-next-app@latest driving-school-chatbot
```

Then run: `cd driving-school-chatbot` and `npm run dev`.

Important project decision

Before using real customer data, confirm the school's approved course details, fee information, booking process, privacy requirements and who will handle incoming requests. Start with sample data for development, then replace it with verified school information.